

Multi-Cloud Wildlife-Image Job-Processing Service: Design

Lucas Huang

1 Overview, Assumptions and Key Decisions

Partners upload a 1–5 GB image batch; each job runs *prep* (CPU) → *species ID* (GPU) → *report* (CPU). Four principles drive the design. (i) One transactional store (PostgreSQL) holds job state, the queue and leases, so there is never a dual-write between a queue and a database. (ii) *Execution is at-least-once, publication is exactly-once*: workers pull fenced leases, write create-only per-attempt objects, and a conditional database update publishes exactly one attempt. (iii) A result becomes *visible* only after a small receipt for it exists in *both* clouds, so a lagging replica can be reconciled without producing a conflicting result. (iv) *Compute is active-active across AWS and GCP, the database writer is active-passive*, and jobs run data-locally to avoid egress. When safety and write availability conflict during an ambiguous partition, safety wins.

Assumption	Value and consequence
Workload	100 jobs/day; burst 100/h; mean batch 3 GB ($\approx$ 1,000 JPEGs); mean job 15 min split 5/6/4 min (prep/ID/report); prep output $\approx$ 5–10% of input. These are sizing hypotheses, to be benchmarked on each VM type.
GPU is the bottleneck	4 GPUs $\times$ 60/6 = 40 jobs/h . A 100-job burst needs $\geq$ 2.5 h of GPU service plus pipeline fill and cold start; $\approx$ 60 jobs still wait at the end of the burst hour $\Rightarrow$ durable queue, ETA shown to users, and acceptance never blocks on capacity. Daily GPU use is $\approx$ 10 of 96 GPU-h ($\approx$ 10%) $\Rightarrow$ GPU pools scale 0–2 per cloud.
CPU sizing	Burst prep 100 $\times$ 5 min $\approx$ 8.3 VMs; report is GPU-bound (40 $\times$ 4 min $\approx$ 2.7 VMs) $\Rightarrow$ $\approx$ 11 of 14 CPU workers at peak, if placement is balanced.
VM cap	Per cloud: 1 on-demand system node (KEDA, CoreDNS, agents) + 0–7 CPU + 0–2 GPU = 10, so 2 + 14 + 4 = 20 . Managed services (RDS/Cloud SQL, EKS/GKE masters, Fargate/Cloud Run, object storage) are assumed outside the cap; if not, the CPU ceiling drops accordingly.
Network	Partner $\rightarrow$ storage burst 300 GB/h $\approx$ 670 Mbps (1.1 Gbps if every batch is 5 GB), so uploads go directly to storage, not through the API. VM $\leftrightarrow$ object store $\geq$ 5 Gbps. AWS $\leftrightarrow$ GCP: 4 IPsec tunnels, but one flow is capped at $\approx$ 1.25 Gbps per tunnel; plan on 1 Gbps effective (to be measured); same-metro RTT $<$ 5 ms.
Pricing (planning figures, not quotes)	Egress $\approx$ \$0.12/GB; GPU VM $\approx$ \$1.2/h; standard object storage $\approx$ \$0.025/GB-month. Always-on 4 GPUs $\approx$ \$3.5k/month vs. on demand ($\approx$ 10 GPU-h/day + 1 warm GPU in business hours) $\approx$ \$0.8k/month, for a cold start of $\approx$ 5 min (to be measured).
Regions	Both clouds in Sydney: widest Australian GPU supply; data stays in Australia (camera traps also photograph people, so images can be personal information). Prefer the same GPU model in both clouds (e.g. NVIDIA L4) where quota allows, because different GPUs give slightly different floating-point results.
Third parties	Partner OIDC IdP; provider-neutral DNS with health checks; a strongly consistent serverless lease store as failover witness; GitHub Actions. Supplied programs are CLI tools (input dir $\rightarrow$ output dir, exit code). Images and model weights ($\approx$ 1 GB) are mirrored into our own registries and buckets before release, so nothing is pulled from public hubs at run time.

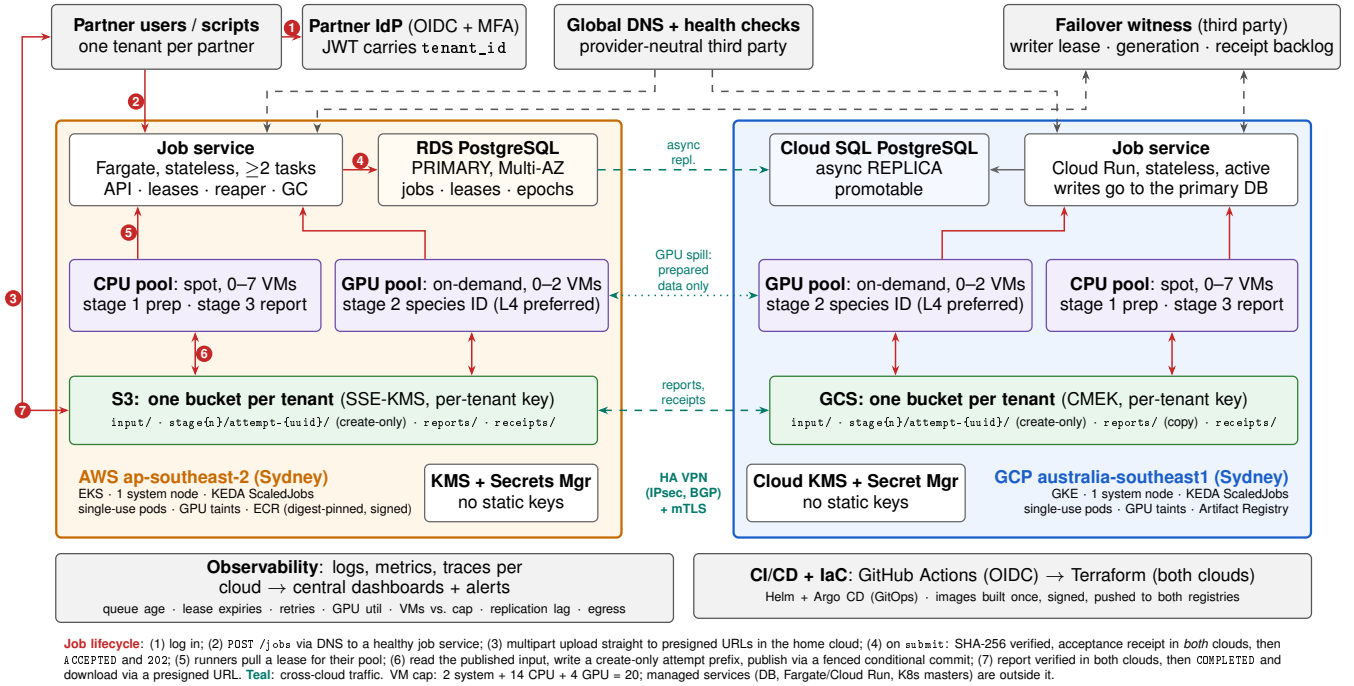


Figure 1: Architecture. Red numbers follow one job; teal links are the only cross-cloud traffic.

2 One Job's Lifecycle

States. Job: UPLOADING → ACCEPTANCE_PENDING → ACCEPTED → RUNNING → COMPLETED; side exits FAILED_NEEDS_REVIEW, CANCELLED. Each stage: QUEUED → RUNNING → PUBLISHING → SUCCEEDED, with RETRY_WAIT and FAILED. (1) The partner signs in through its IdP. The job service validates the JWT (issuer, audience, signature, expiry) and maps the partner to a tenant on the server side. (2) POST /jobs (file list, sizes, SHA-256) picks the home cloud (healthy, earliest estimated finish) and returns per-object multipart-upload URLs scoped to jobs/{id}/input/ in the tenant bucket (15 min, renewable after re-authentication for slow links). (3) The client uploads directly; failed parts are retried individually. (4) POST /jobs/{id}:submit carries a tenant-scoped Idempotency-Key. The service verifies sizes and SHA-256 (multipart ETags are not content hashes) and commits ACCEPTANCE_PENDING with an outbox item. It then writes an immutable *acceptance receipt* (input versions and hashes, home cloud, pinned image and model versions) to both clouds, commits ACCEPTED + PREP_QUEUED, and only then returns 202. A retry with the same key resumes the same job. Lack of capacity only means queueing. (5) A ready CPU runner in the home cloud claims the prep lease. (6) It writes stage1/attempt-{uuid}/ and the coordinator publishes it (§7), which queues stage 2; stage 3 follows the same way. (7) The report is copied and checksum-verified in both clouds and a completion receipt is mirrored. One transaction then records COMPLETED, both object versions and a notification outbox event. Notifications are at-least-once with a stable event ID for de-duplication. GET /jobs/{id} shows stage, queue position and ETA; reports download via 5-minute presigned URLs.

3 Job Coordination and Placement

Acceptance and tracking. The jobs and stage_runs tables are the single source of truth. Each stage row holds status, epoch, attempt_id, owner, lease_expires_at (database clock only) and published_attempt. Load is <1 write/s, so a separate broker (SQS/Pub/Sub) would add a dual-write hazard without benefit.

Worker discovery and assignment (pull). A small *runner agent* is baked into each stage image. It authenticates with workload identity and calls POST /leases with its capabilities (cloud, pool, allocatable memory, GPU model and VRAM, readiness), which the coordinator checks against the pool's configured identity. The coordinator selects an eligible row with SELECT ...FOR UPDATE SKIP LOCKED and, in the same transaction, sets RUNNING, increments epoch, creates a new attempt_id, records the owner and sets a 90s lease. The runner renews the lease every 30s. No registry is needed: a worker exists while it holds or requests leases. Pull also balances load naturally, because idle runners take the next item.

Stage dependencies. A stage becomes QUEUED only after its predecessor is published and that publication is receipted (§8). Consumers read only the *published* manifest, never a prefix listing.

Resources and fairness. At acceptance, each job gets a resource profile from image count and decoded image size (batch bytes alone do not predict peak memory) plus the model's VRAM. A lease matches only a compatible pool, and GPU stages run only on tainted GPU pools. Ordering is *weighted fair share across tenants*, then oldest first. A per-tenant GPU concurrency cap (default 2) stops one partner's 50-job burst from starving the others; unused share may be borrowed.

Placement. Work is placed by *estimated finish time* (queue wait + measured stage runtime + transfer). Prep runs

in the home cloud. Stage 2 *spills* only if $\text{ETA}_{\text{remote}} + t_{\text{transfer}} + 20 \text{ min} < \text{ETA}_{\text{local}}$ (hysteresis). It then copies and verifies the *prepared* output ($\approx 150\text{--}300 \text{ MB}$, $\approx \$0.02\text{--}0.04$) rather than the raw 3 GB batch ($\approx \$0.36$). The report stage stays next to the identification output.

4 VMs, Containers and Orchestration

Provisioning (IaC). Terraform modules per cloud (VPC, VPN/BGP, EKS/GKE, node pools, buckets with lifecycle rules, KMS, primary and standby databases, IAM) plus a global stack (DNS, witness). State is encrypted, locked and access-restricted. GitHub Actions runs `plan` on pull requests and `apply` after review, authenticating through OIDC federation, so CI holds no cloud keys. Helm charts synced by Argo CD keep both clusters on identical, versioned manifests. Schema migrations and failover drills are tested with each release.

Holding the cap. Node pools have `max_size` system 1, CPU 7 and GPU 2 per cloud. The cap must also hold during upgrades and repairs. Surge is therefore disabled (EKS MINIMAL node-group updates, GKE zero-surge), so a node is removed before its replacement is created. Dedicated accounts and projects with restricted provisioning roles stop anyone adding pools outside Terraform, and an alert fires at 19 VMs. An *unreachable* VM still counts: capacity is never reclaimed until its termination is confirmed. CPU workers use spot instances, because a preemption is just another lease expiry. GPU workers and system nodes are on-demand.

Containers. Each stage has one image: the supplied program plus the runner agent and its libraries. Images are built once in CI, scanned, signed (cosign), pushed to both registries and deployed *by digest*. The GPU image pins CUDA and framework versions and a tested host-driver range. Model weights are *not* baked into the image (the same pattern as my A1 SmartPark deployment): an init container fetches `model@version` from our bucket, verifies its SHA-256, and stores it in a node-local, read-only, model-only cache. Tenant data never enters that cache.

Isolation and scaling. KEDA *ScaledJobs* launch one single-use pod per eligible queued lease, and count active and pending pods to avoid over-launching. A pod requests a lease only after its startup checks pass, runs exactly one attempt and exits; an empty pull exits at once. Every tenant job therefore gets a fresh pod with its own scratch `emptyDir`. Requests cover CPU, RAM, ephemeral disk and GPU, which places one attempt per worker VM while leaving room for node agents. The cluster autoscaler works within the pool bounds, and GPU pools scale to zero after 10 min idle (one stays warm in business hours). Pods run as non-root with a read-only root filesystem, all capabilities dropped and seccomp. NetworkPolicy is default-deny: egress is allowed only to the coordinator, identity, DNS, model and storage endpoints. This sharply reduces, but cannot eliminate, the risk of a third-party program exfiltrating data.

5 Storage and ML Reproducibility

Data	Store	Durability, lifecycle and cleanup
Inputs	S3/GCS, bucket + key per tenant, home cloud	Versioned. Deleted 30 days after the job ends (longer while under review); partners keep originals. Not replicated cross-cloud ($\approx 9 \text{ TB/month} \approx \$1,080$ egress), so a home-storage outage pauses those jobs.
Intermediates	same bucket, <code>stage{n}/attempt-{uuid}/</code>	<i>Create-only</i> writes (S3 <code>If-None-Match</code> , GCS <code>ifGenerationMatch=0</code>); <code>_MANIFEST.json</code> (object versions + SHA-256) is written last. GC checks DB references and leases, deletes unpublished attempts after 24 h and published ones 7 days after the job ends, and pauses whenever the database is unavailable.
Reports	both clouds, <code>reports/</code>	Verified in both clouds before COMPLETED ; versioned; workers cannot delete them; kept 1 year by policy; moved to Infrequent Access after 30 days. Object lock is used only if the agreed policy requires it, so deletion requests can still be honoured.
Metadata	RDS PostgreSQL (Multi-AZ) → Cloud SQL replica	PITR 14 days, tested backups, async logical replica in GCP (lag typically $<5 \text{ s}$). Receipts in both clouds cover the lag (§8).

Provenance. Each published stage run stores an immutable record: input and output manifest hashes, attempt and epoch, image digest, program version, model name/version/SHA-256, framework/CUDA/driver versions, GPU model, instance type, region, timings and exit code. The report embeds a provenance digest. The same image and model can still differ slightly in floating point across GPU models, so the hardware is recorded and pinned when bitwise reproducibility is required. Images and models referenced by retained reports are kept for the report’s lifetime.

6 Health and Recovery

Readiness. A startup check verifies the model checksum, GPU allocation and a CUDA self-test, free scratch space, and that storage and the coordinator are reachable; only then does the runner claim work. Node health comes from the kubelet and GPU (Xid) errors.

Crashes. A non-zero exit or OOM is reported by the agent. A lost pod, VM, preemption or partition appears as missing heartbeats. The reaper locks the row, confirms expiry on the database clock, marks the attempt **ABANDONED** and requeues the stage; the next claim increments the epoch.

Stalls. Heartbeats prove only that the agent is alive, not that useful work is happening, so they also carry progress: output bytes, program progress where available, and GPU utilisation for stage 2 (an idle CPU does not mean a GPU

kernel has stalled). No progress for 10 min, or exceeding $2\times$ the historical p99 for the size class and hardware, kills and requeues the attempt. The agent also stops itself when it cannot renew, but safety never depends on that (§7).

Repeated failures are classified: (a) *transient program failures* are retried at most twice with exponential backoff and jitter, and an OOM moves to a larger-memory pool; (b) *deterministic input or configuration errors* are not retried and go to the partner with a reason; (c) *infrastructure failures* have a separate budget (5 attempts or 24 h) and lower the node’s health score, and 2 failures from different jobs within 1 h quarantine the node. An exhausted budget leaves the job **FAILED_NEEDS_REVIEW**: its data is kept, an alert fires, and the queue continues. Cancellation invalidates leases and blocks publication.

Logs and metrics. Structured logs carry **tenant**, **job**, **stage**, **attempt**, **epoch**, **trace ID**, **image digest**, **model hash** and failure category, never image contents or secrets. Metrics and alerts: oldest queue age, stage latency by size and hardware, GPU utilisation and VRAM, lease-expiry and retry rates, tenant fairness, object-copy failures, receipt-mirror backlog, replication lag, VM count vs. cap, egress bytes and stalled finalisation.

7 Failure Scenario: GPU VM Lost After Writing Its Output

Time	Event and state change
t_0	G1 has claimed stage 2 of job J: epoch=7 , attempt A. It writes stage2/attempt-A/ with create-only writes, then a manifest of object versions and SHA-256. Only the coordinator can move the publication pointer.
t_1	G1 becomes unreachable before committing; its heartbeats stop. Its storage credentials may outlive the lease (AWS STS sessions last ≥ 15 min) but can write only attempt-A/ , which nobody reads unless it is published. Credentials are therefore <i>not</i> the safety mechanism.
$t_1+90\text{ s}$	The reaper locks the row, confirms expiry on the DB clock, marks A ABANDONED and sets the stage QUEUED ; epoch stays 7. A commit from G1 now fails, because the stage is not RUNNING .
t_2	G2 claims the stage (epoch=8 , attempt B), writes attempt-B/ and commits: UPDATE stage_runs SET status='PUBLISHING', published_attempt='B' WHERE job=J AND stage=2 AND attempt_id='B' AND owner='G2' AND epoch=8 AND status='RUNNING' AND lease_expires_at > now() → 1 row . Once the publication receipt is mirrored, a second transaction sets SUCCEEDED and queues stage 3. An identical repeated commit returns the same success.
t_3	G1 returns and commits with epoch=7 , attempt A → 0 rows → 409 STALE_LEASE . The agent discards its work and exits. Kubernetes has already deleted the pod, so the kubelet kills it on reconnect.
$t_3+24\text{ h}$	GC confirms that A is unreferenced and deletes attempt-A/ .

Why there is no duplicate or conflicting result. (a) Attempts never share a path, and writes are create-only, so G1 cannot overwrite G2. (b) Exactly one attempt can satisfy the predicate. (c) Consumers follow the published pointer only, so an orphan is invisible. (d) **COMPLETED** and the outbox event are written in one transaction, and notifications carry a stable ID. (e) Lease time is the database’s clock, so drift on G1 cannot help it. The guarantee is one *authoritative published result*, not one physical execution. *Considered and rejected:* adopting G1’s verified orphan would save ≈ 6 GPU-min and is safe with create-only objects, but it adds a second publication path, so we rerun instead.

8 Multi-Cloud Operation

Strategy: active-active compute, active-passive database writer (warm standby). Both clouds run job services and workers every day, so failover capacity is always warm and tested. The only singleton is the PostgreSQL writer (AWS); GCP’s job service writes to it over the VPN, and GCP holds an async replica. A multi-master database would need ≥ 3 regions for quorum and a further third-party dependency, which is too much for 100 jobs/day.

Connectivity and egress. AWS Site-to-Site VPN ↔ GCP HA VPN (4 tunnels, BGP, non-overlapping CIDRs), with TLS to the database and mTLS between services. Cross-cloud traffic consists of DB replication, GCP-to-primary writes, receipt and report copies, and rare spills. A copier service sends these through the VPN to private storage endpoints. Partners’ presigned-URL traffic never crosses clouds. Data-local placement cuts cross-cloud bytes from ≈ 9 TB/month ($\approx \$1,080$) to ≈ 30 GB/month ($\approx \4), plus spills; a dedicated interconnect is not justified at this volume.

Visibility rule (replication safety). Three small immutable receipts (*acceptance*, *stage publication* and *completion*) are mirrored to both clouds *before* the matching state becomes visible, i.e. before the 202, before the next stage is queued, and before the partner is notified. Each transition is two-step in the database (e.g. **PUBLISHING** → **SUCCEEDED**) with the mirror in between, driven by an outbox worker, so a crash between the steps only delays them. Every fact anyone has seen therefore exists outside the primary, even when replication lags. The cost is one cross-cloud object write (< 1 s) per stage.

Dynamic failover (AWS unreachable). (1) *Detect and route:* DNS health checks from several locations and GCP probes fail for 3 min, and DNS sends all API traffic to GCP. Status (labelled “last known”) and report downloads, which rely on mirrored receipts, continue immediately. (2) *Fence the old writer:* the AWS job services renew a 60 s *writer lease* (generation g) in the witness. Every job service, in either cloud, opens a write transaction only while that lease has ≥ 15 s left, and statements time out after 5 s, so after expiry plus a clock-skew margin nothing can still be writing to the old primary. Each renewal also records the receipt-mirror backlog. (3) *Promote only when safe:* if

the lease has expired *and* its last renewal reported backlog = 0, GCP promotes Cloud SQL as generation $g+1$ and reconciles from receipts. Missing accepted jobs are re-created, and missing publications are restored with the *same* attempt IDs, so no stage is ever re-published differently. Unreceipted in-flight work was never visible and is simply rerun. Otherwise (backlog > 0, or the witness is down) the system stays read-only until an operator decides; fencing is never bypassed.

Keeps operating: new submissions (homed in GCP), GCP-homed jobs, status, and every report. **Paused, not lost:** AWS-homed jobs whose inputs are in S3. Capacity drops to 10 VMs (20 GPU jobs/h), because the unreachable VMs still count against the cap. Targets: read RTO ≈ 3 min, write RTO ≈ 5 min when the gate passes, RPO = 0 for visible results. **GCP outage:** AWS stays primary in degraded mode. It accepts and publishes with local receipts and a non-zero backlog, which blocks automatic promotion until GCP returns and the backlog drains. GCP-homed jobs pause. **Link-only partition** (both clouds healthy): AWS still renews the lease, so nothing is promoted; GCP serves reads, and its runners finish current attempts but cannot publish until the link returns. **Failback:** the old primary is rebuilt as a replica and switched over in a quiet window. Requests carrying generation g are refused.

9 Cloud Security

Users. Partners sign in through OIDC with MFA; the JWT's issuer, audience, signature and expiry are validated, and the tenant is mapped server-side. Automation uses scoped OAuth client credentials. **Services.** Workloads use workload identity (EKS Pod Identity, GKE Workload Identity), so no static cloud keys exist. Internal APIs use mTLS, and database connections use TLS.

Access control (defence in depth). (1) The tenant comes from the token, never from the request body, and ownership is checked on every call. (2) PostgreSQL row-level security: `SET LOCAL app.tenant_id` in each transaction, application roles that are not table owners and lack `BYPASSRLS`, and a separate restricted role for the scheduler. (3) One bucket and one KMS key per tenant per cloud; both report copies and receipts are protected by the same tenant checks. (4) Runners get only the exact input and model reads and attempt-prefix writes they need. Because STS sessions last at least 15 min, the coordinator's lease, not the credential, is the authority. (5) Presigned URLs are per object and short-lived. Retiring a tenant's key crypto-shreds its data once retention and backup obligations are reconciled.

Secrets. Secrets live in AWS Secrets Manager / GCP Secret Manager, are read at run time through workload identity and are rotated; databases use IAM authentication. CI runs secret scanning. **Network isolation.** Workers and databases are private (no public IPs) and use private storage endpoints. The API sits behind a WAF and rate limits, and admission control (Kyverno / Binary Authorization) admits only signed images. **Encryption.** TLS 1.2+ externally, mTLS internally, and IPsec between clouds. At rest: SSE-KMS/CMEK with per-tenant keys, and KMS-encrypted databases, disks and backups. Audit trail: CloudTrail data events, GCS data-access logs, and DB audit of every state transition.

10 Trade-offs and Limitations

Safety over availability: in an ambiguous partition the system may stay read-only rather than risk two writers. **Receipts** add <1 s per stage in exchange for RPO = 0 on visible results. **Inputs are not replicated:** this saves $\approx \$1k$ /month but pauses home-cloud jobs during an outage. **Scale-to-zero GPUs** save $\approx \$2.7k$ /month for a ≈ 5 min cold start. **Third-party witness and DNS:** if either fails, failover falls back to an operator decision.